University of Applied Sciences
HOCHSCHULE EMDEN·LEER

SLAM Algorithms

Srikanth Nanda

27. November 2025

1 FastSLAM

2 Active SLAM

3 MonoSLAM

4 ORB-SLAM

5 Visual SLAM Surveys

6 SLAM Overview

7 SLAM Algorithms Code Implementation

8 Quellen

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

FastSLAM

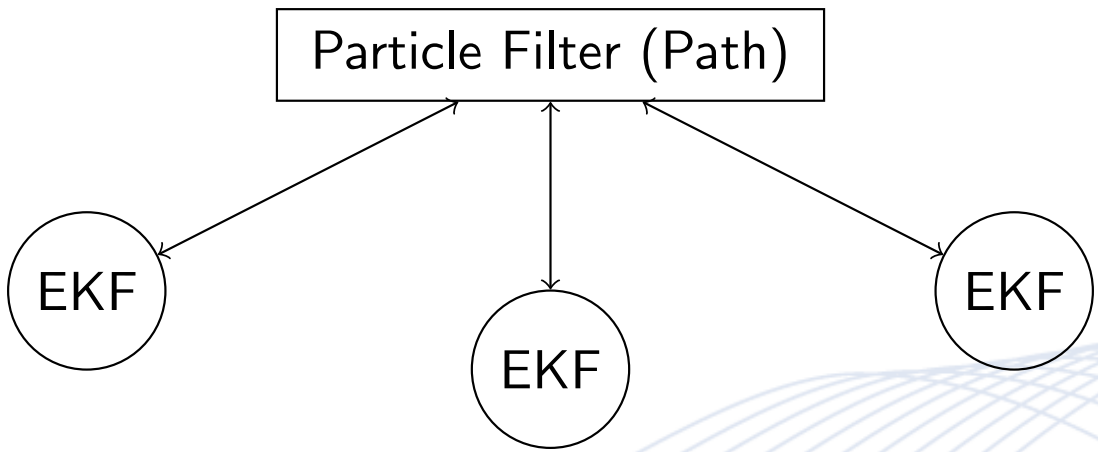
Idea I

Based on the AAAI-02 paper, FastSLAM introduces a **factored representation of the SLAM (Simultaneous Localization and Mapping) posterior**:

$$p(x_{1:t}, m \mid z_{1:t}, u_{1:t}) = p(x_{1:t} \mid z_{1:t}, u_{1:t}) \prod_k p(m_k \mid x_{1:t}, z_{1:t})$$

This exploits the conditional independence between landmarks and robot trajectory. The algorithm combines a **particle filter for robot paths** with **independent EKFs for landmarks**. A tree-based structure enables **logarithmic map updates**, achieving scalability to tens of thousands of features.Montemerlo u. a., 2002

Conceptual Structure I



University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Hyperparameters I

- Number of particles
- Motion and measurement noise
- Landmark initialization covariance
- Tree resolution for map updatesMontemerlo u. a., 2002

Applications I

Large-scale outdoor SLAM, robot navigation in high-feature environments.Montemerlo u. a., 2002

Active SLAM

Idea I

From the IEEE/ASME T-Mech paper, Active SLAM frames SLAM as a **dual-mode MPC problem**:

Mode 1: SLAM-uncertainty reduction

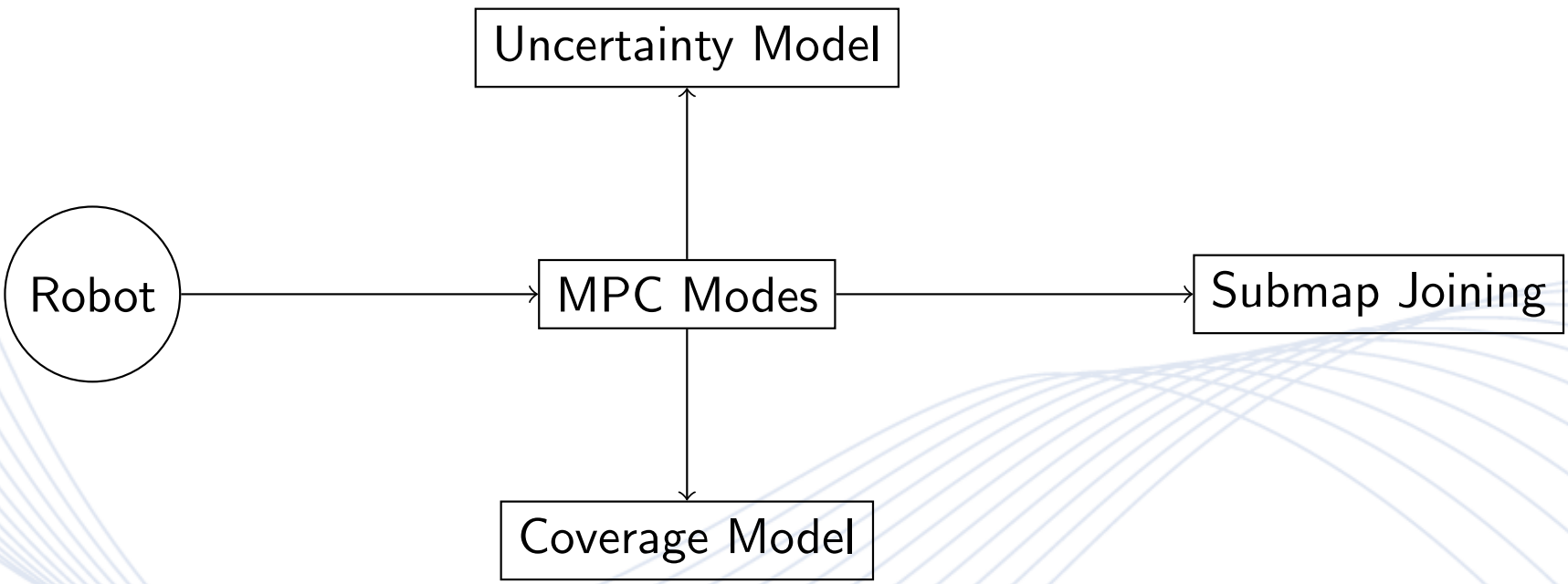
Uses convex relaxation, graph topology, and Fisher Information eigenvalue bounds to approximate future uncertainty. Chen, Huang und Fitch, 2018

Mode 2: Coverage

Sequential quadratic programming ensures environment coverage while maintaining safe trajectories.

The authors employ **Linear SLAM** (submap joining) for fast covariance recovery and scalable planning. Chen, Huang und Fitch, 2018

Conceptual Structure I



University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Hyperparameters I

- MPC horizon
- Convex relaxation parameters
- SLAM measurement noise
- Control bounds Chen, Huang und Fitch, 2018

Applications I

Autonomous UAV exploration, obstacle-aware navigation, area scanning.
Chen, Huang und Fitch, 2018

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

MonoSLAM

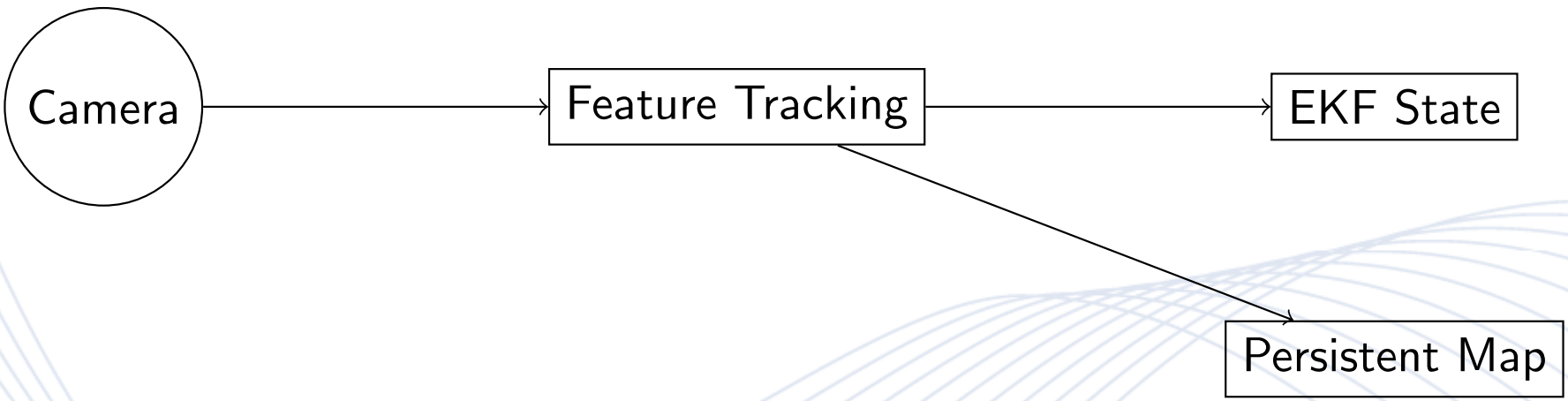
Idea I

The PAMI 2007 paper presents the first real-time monocular SLAM with drift-free performance. MonoSLAM uses:

- **EKF joint state**: camera pose + 3D features
- **Active measurement search**: predicts feature uncertainty ellipses
- **General 3D motion model** for smooth tracking
- **Inverse-depth feature initialization**

The emphasis is on robust localization for tasks such as humanoid robot control and augmented reality.Davison u. a., 2007

Conceptual Structure I



University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Hyperparameters I

Process noise, measurement noise, feature search windows, initialization depth.Davison u. a., 2007

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Applications I

Augmented reality, humanoid robots, embedded monocular SLAM.Davison u. a., 2007

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

ORB-SLAM

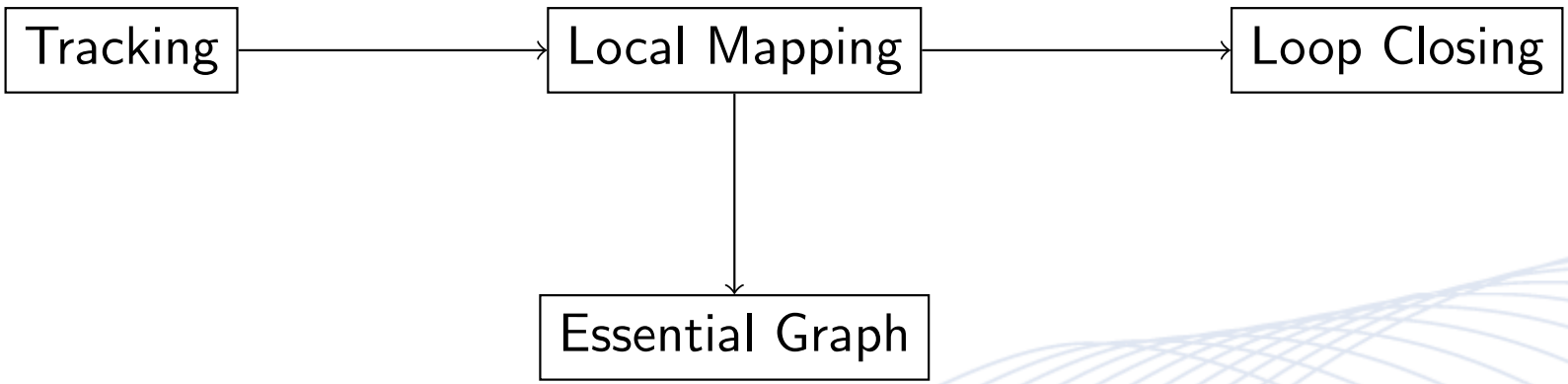
Idea I

According to the IEEE Transactions on Robotics paper, ORB-SLAM is designed for real-time, robust large-scale operation:

- **ORB features** for all SLAM tasks
- **Covisibility graph** for local mapping
- **Essential graph** for global optimization
- **Automatic initialization** via model selection
- **Survival-of-the-fittest** keyframe policy Mur-Artal, Montiel und Tardós, 2015

It achieves state-of-the-art accuracy across 27 sequences.

Conceptual Structure I



Hyperparameters I

Number of ORB features, BA window size, keyframe thresholds, loop closure limits. Mur-Artal, Montiel und Tardós, 2015

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Applications I

AR/VR localization, robotics, drone navigation, lifelong SLAM.
Mur-Artal, Montiel und Tardós, 2015

Visual SLAM Surveys

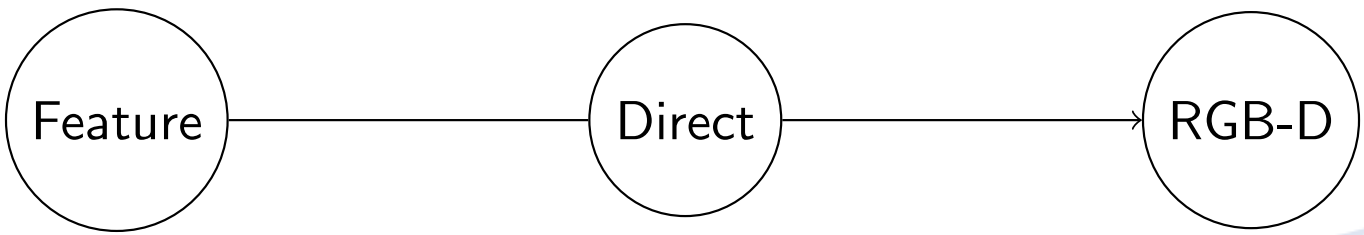
Idea I

Document insights reveal three dominant categories:

- **Feature-based SLAM** (ORB-SLAM, MonoSLAM)
- **Direct SLAM** (LSD-SLAM, DSO)
- **RGB-D SLAM** (ElasticFusion, Kintinuous) Barros u. a., 2022; Taketomi, Uchiyama und Ikeda, 2017

Major evaluation dimensions include: tracking robustness, drift handling, loop closure, computational efficiency. Taketomi, Uchiyama und Ikeda, 2017

Conceptual Structure I



University of Applied Sciences
HOCHSCHULE EMDEN·LEER

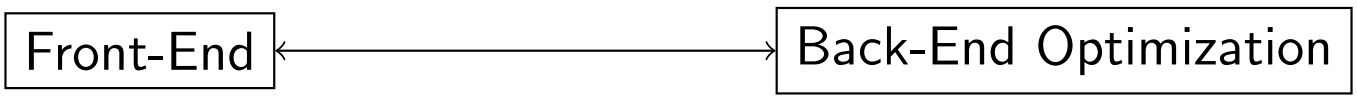
SLAM Overview

Idea I

The overview document explains SLAM in terms of:

- **Front-end**: feature extraction, scan matching, data association
- **Back-end**: optimization (EKF, smoothing, pose graphs)
- **Sensors**: LiDAR, camera, IMU, GNSS integration
- **Failure modes**: drift, wrong associations, poor initialization Alsadik und Karam, 2021

Conceptual Structure I



University of Applied Sciences
HOCHSCHULE EMDEN·LEER

SLAM Algorithms Code Implementation

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Python Implementation SLAM Algorithms I

```
#!/usr/bin/env python3
"""
/**
 * @file SLAM.py
 * @brief Minimal implementations of FastSLAM, Active SLAM, MonoSLAM (EKF-SLAM),
 *        and ORB-style feature pipeline for teaching & presentation purposes.
 * @author Srikanth
 * @date 2025
 *
 * This module includes:
 *   - FastSLAM: Rao-Blackwellized particle filter + landmark EKFs
 *   - Active SLAM: Utility-driven action selection (simplified)
 *   - MonoSLAM / EKF-SLAM: Joint state EKF for camera + features
 *   - ORB-SLAM style pipeline: ORB detection, matching, pose estimation
```

Python Implementation SLAM Algorithms II

```
15  *
16  * Dependencies:
17  *   - Python3, OpenCV, NumPy
18  */
19 """
20
21 import numpy as np
22 import cv2
23 import math
24 import random
25
26 ################################################################################
27 # 1. FASTSLAM (Simplified RBPF)
28 ################################################################################
29
30 class LandmarkEKF:
```

Python Implementation SLAM Algorithms III

```
31      """
32      /**
33       * @class LandmarkEKF
34       * @brief Extended Kalman Filter for a single landmark.
35       *
36       * This class maintains the mean and covariance of one
        landmark
37       * and provides an EKF update step.
38       */
39      """
40      def __init__(self, mu, Sigma):
41          """/**
42          * @brief Construct a new LandmarkEKF object.
43          * @param mu Initial mean (2D vector)
44          * @param Sigma Initial covariance (2x2 matrix)
45          */"""
46          self.mu = mu
47          self.Sigma = Sigma
48
49      def update(self, z, H, R):
```

Python Implementation SLAM Algorithms IV

```
50          """/**
51          * @brief Perform EKF measurement update.
52          * @param z Measurement vector
53          * @param H Observation Jacobian
54          * @param R Measurement noise covariance
55          */"""
56          S = H @ self.Sigma @ H.T + R
57          K = self.Sigma @ H.T @ np.linalg.inv(S)
58          self.mu = self.mu + K @ (z - H @ self.mu)
59          self.Sigma = (np.eye(len(self.mu)) - K @ H) @ self.
    Sigma
60
61  class FastSLAMParticle:
62      """
63      /**
64       * @class FastSLAMParticle
65       * @brief One particle in the Rao Blackwellized particle
         filter.
66       *
67       * Contains robot pose and a map of landmark EKFs.
```

Python Implementation SLAM Algorithms V

```
68          */
69         """
70         def __init__(self, pose):
71             """/**
72                 * @brief Construct a new FastSLAMParticle with an
           initial pose.
73                 * @param pose 3D vector [x, y, theta]
74                 */"""
75             self.pose = pose
76             self.landmarks = {}
77
78         def predict(self, u, noise):
79             """/**
80                 * @brief Apply motion model with noise.
81                 * @param u Control input [v, w]
82                 * @param noise Noise standard deviations [nv, nw]
83                 */"""
84             v, w = u
85             noisy_v = v + np.random.normal(0, noise[0])
86             noisy_w = w + np.random.normal(0, noise[1])
```

Python Implementation SLAM Algorithms VI

```
87
88          x, y, th = self.pose
89          x += noisy_v * math.cos(th)
90          y += noisy_v * math.sin(th)
91          th += noisy_w
92          self.pose = np.array([x, y, th])
93
94  class FastSLAM:
95      """
96      /**
97       * @class FastSLAM
98       * @brief Rao Blackwellized SLAM with independent
    landmark EKFs.
99       */
100     """
101     def __init__(self, num_particles=50):
102         """/**
103         * @brief Construct FastSLAM with given number of
    particles.
104         * @param num_particles Number of particles
```

Python Implementation SLAM Algorithms VII

```
105         */"""
106         self.particles = [FastSLAMParticle(np.zeros(3)) for
    _ in range(num_particles)]
107
108     def predict(self, u):
109         """/**
110         * @brief Predict step for all particles.
111         * @param u Control input [v, w]
112         */"""
113         for p in self.particles:
114             p.predict(u, noise=[0.01, 0.01])
115
116     def update(self, z, landmark_id):
117         """/**
118         * @brief Update all particles with a measurement.
119         * @param z Measurement of landmark position
120         * @param landmark_id ID of the observed landmark
121         */"""
122         for p in self.particles:
123             if landmark_id not in p.landmarks:
```

Python Implementation SLAM Algorithms VIII

```
124                     p.landmarks[landmark_id] = LandmarkEKF(mu=np
    .array([z[0], z[1]]),
125                                                            Sigma
    =np.eye(2) * 1.0)
126
127             lm = p.landmarks[landmark_id]
128             H = np.eye(2)
129             R = np.eye(2) * 0.05
130             lm.update(z, H, R)
131
132 #
    ##################################################################

133 # 2. ACTIVE SLAM (Simplified Utility-Based Planning)
134 #
    ##################################################################


135
136 class ActiveSLAM:
137     """
```

Python Implementation SLAM Algorithms IX

```
138     /**
139      * @class ActiveSLAM
140      * @brief Simplified utility-driven SLAM planner.
141      */
142     """
143     def __init__(self):
144         """/**
145         * @brief Construct ActiveSLAM with predefined action
         set.
146         */"""
147         self.actions = [
148             np.array([0.2, 0.0]),    # forward
149             np.array([0.0, 0.4]),    # rotate left
150             np.array([0.0, -0.4])    # rotate right
151         ]
152 
153     def info_gain(self, Sigma):
154         """/**
155         * @brief Compute expected information gain.
156         * @param Sigma Current covariance
```

Python Implementation SLAM Algorithms X

```
157         * @return Log determinant score
158         */"""
159         return -np.log(np.linalg.det(Sigma) + 1e-9)
160 
161     def choose_action(self, Sigma):
162         """/**
163         * @brief Choose action that maximizes information
    gain.
164         * @param Sigma Current covariance
165         * @return Selected control action
166         */"""
167         best = None
168         best_u = None
169         for u in self.actions:
170             pred_sigma = Sigma - np.eye(2) * 0.001
171             score = self.info_gain(pred_sigma)
172             if best is None or score > best:
173                 best = score
174                 best_u = u
175         return best_u
```

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Python Implementation SLAM Algorithms XI

```
176
177 #
        ##############################################################################
178 # 3. Monocular EKF-SLAM (MonoSLAM-like)
179 #
        ##############################################################################
180
181 class EKFSLAM:
182     """
183     /**
184      * @class EKFSLAM
185      * @brief MonoSLAM-like EKF joint state estimator.
186      */
187     """
188     def __init__(self, max_landmarks=20):
189         """/**
190         * @brief Construct EKF-SLAM.
191         * @param max_landmarks Maximum number of landmarks
```

Python Implementation SLAM Algorithms XII

```
192         */"""
193         self.mu = np.zeros(3)
194         self.Sigma = np.eye(3) * 0.01
195         self.max_landmarks = max_landmarks
196         self.landmarks = {}
197 
198     def predict(self, u):
199         """/**
200         * @brief EKF predict step.
201         * @param u Control input [v, w]
202         */"""
203         v, w = u
204         x, y, th = self.mu
205 
206         x_new = x + v * math.cos(th)
207         y_new = y + v * math.sin(th)
208         th_new = th + w
209 
210         self.mu = np.array([x_new, y_new, th_new])
211         self.Sigma += np.eye(3) * 0.001
```

Python Implementation SLAM Algorithms XIII

```
212
213     def update(self, landmark_id, z):
214         """/**
215         * @brief EKF update with a landmark measurement.
216         * @param landmark_id ID of the observed landmark
217         * @param z 2D measurement vector
218         */"""
219         if landmark_id not in self.landmarks:
220             self.landmarks[landmark_id] = LandmarkEKF(mu=np.
    array(z),
221                                                       Sigma=
    np.eye(2) * 1.0)
222         self.landmarks[landmark_id].update(
223             z=z,
224             H=np.eye(2),
225             R=np.eye(2) * 0.05
226         )
227
```

Python Implementation SLAM Algorithms XIV

```
228 #
        ##############################################################################
229 # 4. ORB-SLAM-style Feature Detection + Matching
230 #
        ##############################################################################
231
232 class ORB_SLAM_Pipeline:
233     """
234     /**
235      * @class ORB_SLAM_Pipeline
236      * @brief Simplified ORB front-end pipeline.
237      */
238     """
239     def __init__(self):
240         """/**
241         * @brief Construct ORB-SLAM pipeline with ORB
        extractor and BF matcher.
242         */"""
```

Python Implementation SLAM Algorithms XV

```
243         self.orb = cv2.ORB_create(2000)
244         self.matcher = cv2.BFMatcher(cv2.NORM_HAMMING)
245 
246     def extract(self, frame):
247         """/**
248         * @brief Extract ORB keypoints and descriptors.
249         * @param frame Input image
250         * @return (keypoints, descriptors)
251         */"""
252         kp, des = self.orb.detectAndCompute(frame, None)
253         return kp, des
254 
255     def match(self, des1, des2):
256         """/**
257         * @brief Match descriptors using KNN + Lowe ratio
    test.
258         * @param des1 Descriptors from image 1
259         * @param des2 Descriptors from image 2
260         * @return List of good matches
261         */"""
```

Python Implementation SLAM Algorithms XVI

```
262         raw = self.matcher.knnMatch(des1, des2, k=2)
263         good = []
264         for m, n in raw:
265             if m.distance < 0.75 * n.distance:
266                 good.append(m)
267         return good
268 
269     def estimate_pose(self, kp1, kp2, matches):
270         """/**
271         * @brief Estimate pose using matched keypoints and
    homography.
272         * @param kp1 Keypoints from image 1
273         * @param kp2 Keypoints from image 2
274         * @param matches Filtered matches
275         * @return Homography matrix or None
276         */"""
277         if len(matches) < 8:
278             return None
279 
```

Python Implementation SLAM Algorithms XVII

```
280         pts1 = np.float32([kp1[m.queryIdx].pt for m in
    matches])
281         pts2 = np.float32([kp2[m.trainIdx].pt for m in
    matches])
282 
283         H, mask = cv2.findHomography(pts1, pts2, cv2.RANSAC)
284         return H
```

Listing 1: Python Implementation SLAM Algorithms

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Vielen Dank

für Ihre Aufmerksamkeit

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Quellen

References I

- Alsadik, Bashar und Samer Karam (2021). "The Simultaneous Localization and Mapping (SLAM)–An Overview". In: *Surveying and Geospatial Engineering Journal* 2.1, S. 1–12.
- Barros, Andréa Macario u. a. (2022). "A Comprehensive Survey of Visual SLAM Algorithms". In: *Robotics* 11.1, S. 24.
- Chen, Yongbo, Shoudong Huang und Robert Fitch (2018). "Active SLAM for Mobile Robots with Area Coverage and Obstacle Avoidance". In: *IEEE/ASME Transactions on Mechatronics*.
- Davison, Andrew J. u. a. (2007). "MonoSLAM: Real-Time Single Camera SLAM". In: *IEEE Trans. Pattern Analysis and Machine Intelligence* 29.6, S. 1052–1067.
- Montemerlo, Michael u. a. (2002). "FastSLAM: A Factored Solution to the Simultaneous Localization and Mapping Problem". In: *AAAI Conference on Artificial Intelligence*.

References II

 Mur-Artal, Raul, J. M. M. Montiel und Juan D. Tardós (2015). "ORB-SLAM: A Versatile and Accurate Monocular SLAM System". In: *IEEE Transactions on Robotics*.

 Taketomi, Takafumi, Hideaki Uchiyama und Sei Ikeda (2017). "Visual SLAM Algorithms: A Survey from 2010 to 2016". In: *IPSJ Transactions on Computer Vision and Applications* 9, S. 16.